ChatGPT said:

Time-Series Databases (TSDBs): DSA, LLD, and HLD Perspectives

A **time-series database (TSDB)** is a database optimized for storing, indexing, querying, and retaining data points associated with timestamps. Prometheus is a prominent example, designed primarily for monitoring applications, infrastructure, and distributed systems.

From a software engineering and system design interview perspective, the key insight is that **a time-series database optimizes for time-oriented writes, range queries, compression, and retention**, rather than treating every data point like an ordinary transactional database row.

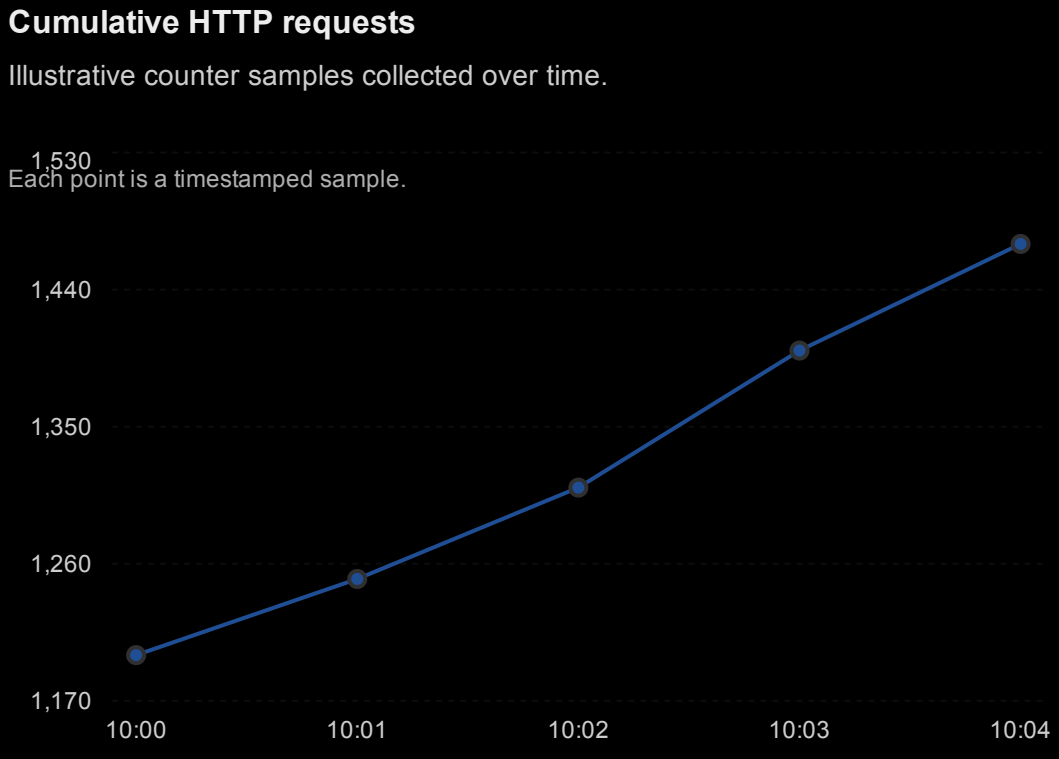
We'll approach this at three levels:

- **DSA (Data Structures and Algorithms):** How samples are indexed, stored, compressed, and queried efficiently.
- **LLD (Low-Level Design):** How to design the classes, interfaces, APIs, and components of a TSDB.
- **HLD (High-Level Design):** How to architect a scalable monitoring platform using Prometheus-like components.

1. Fundamentals: What is a time series?

Consider a Java service exposing the number of HTTP requests it has processed.

Metric: `http_requests_total`



A time series is a sequence of samples:

$$S = \{(t_1, v_1), (t_2, v_2), \ldots, (t_n, v_n)\}$$

Where:

- t_i: timestamp at which the sample was recorded.
- v_i: value observed at that time.
- n: number of samples.

In Prometheus, a series is identified by its **metric name and complete set of labels**.

For example:

Plain text

```
http_requests_total{
    service="payment",
    method="POST",
    status="200"
}
```

The following are two different time series:

Plain text

```
http_requests_total{service="payment", status="200"}
http_requests_total{service="payment", status="500"}
```

Even though they share a metric name, their label sets differ.

A simplified logical sample is:

Java

```
record Sample(
    long timestampMillis,
    double value
) {}
```

The actual stored representation is typically more compact than a Java object per sample.

Why not use PostgreSQL or MongoDB?

You can store time-series data in either, but TSDBs specialize in workloads such as:

- Continuous ingestion of small samples.
- Queries over time windows, such as the last five minutes.
- Aggregations over millions or billions of samples.
- Compression of repetitive timestamp/value patterns.
- Retention and deletion of old data.
- Efficient retrieval of individual series and groups of series.

A crucial distinction: Prometheus is not merely a storage engine. It is a monitoring system that includes metric scraping, querying, rules, and alerting, alongside its time-series storage.

2. DSA perspective: How to store and query time-series data efficiently

At the DSA level, a TSDB is a combination of several data structures and algorithms optimized for different operations.

2.1 Core data structures

1. Hash map — series lookup

Map a canonicalized metric name and label set to a series identifier.

```
Plain text
SeriesKey → SeriesID

(http_requests_total,
 method=POST, status=200) → 42
```

Expected lookup complexity: $O(1)$, assuming a good hash distribution.

2. Inverted index — label filtering

Index label names and values to identify matching series without scanning every series.

```
Plain text
status="500" → {SeriesID 12, 25, 42}
service="payment" → {SeriesID 12, 42, 51}
```

Intersecting candidate sets can dramatically reduce the amount of data examined.

3. Append-oriented chunks — sample storage

Store timestamp/value pairs in contiguous or compact encoded sequences rather than allocating a heavyweight object for every sample.

```
Plain text

Series 42:
(1000, 12.5), (2000, 13.0), (3000, 12.8)
```

This improves cache locality and reduces memory overhead.

4. Time-partitioned blocks — historical storage

Organize samples into blocks covering time ranges. Query engines can skip blocks that do not overlap the requested time interval.

These are conceptual structures, not a claim that Prometheus implements each one exactly as illustrated. Its actual storage engine uses a series index, chunk storage, a WAL, and time-based blocks.

Prometheus +1

2.2 Why the B-tree is not always the best answer

In a relational database, you might create an index on `(metric_name, timestamp)`. That can work well, but it is not the only design.

A TSDB often benefits from separating two concerns:

1. **Which series match the query?** Use the series index and label postings.
2. **Which samples within those series match the time range?** Use time-ordered chunks and block metadata.

For example, consider this query:

```
Plain text

Find HTTP 500 errors from 10:00 to 10:05
for service="payment".
```

The execution plan can conceptually do the following:

1. Find series matching `service="payment"`.
2. Find series matching `status="500"`.
3. Intersect the series IDs.
4. Locate chunks overlapping the requested time range.
5. Decode the relevant samples.
6. Apply the requested aggregation.

This is similar to a database query plan that uses indexes to reduce the candidate rows before performing expensive operations.

Choosing the right structure

Data structure	Best use	Main trade-off
Hash map	Exact series lookup	Poor for ordered range scans
Inverted index	Filter by labels	Index memory grows with series and label values
Sorted arrays / encoded chunks	Sequential sample scans	Inserts and updates require careful handling
B-tree	Ordered lookup and range queries	Tree traversal and page overhead
LSM tree	High write throughput	Compaction and write amplification
Time-partitioned blocks	Historical range queries and retention	Requires block management and compaction

Interview insight: Do not say every TSDB uses an LSM tree or B-tree. Different systems choose different storage engines. Prometheus's local TSDB, for example, is based on a head containing recent data, a write-ahead log, and immutable time-based blocks with indexes and chunks.

Prometheus +1

2.3 Algorithm: Efficient time-range queries

Suppose a series contains one million samples sorted by timestamp, and we need the samples in the interval $[t_s, t_e]$.

A naive algorithm scans all samples:

$$T(n) = O(n)$$

A better algorithm uses binary search to find the first sample at or after t_s, then scans until t_e.

$$T(n, k) = O(\log n + k)$$

Where k is the number of samples returned.

Java implementation:

Java

```
import java.util.ArrayList;
import java.util.List;

record Sample(long timestampMillis, double value) {}

static List<Sample> rangeQuery(
        List<Sample> samples,
        long startInclusive,
        long endInclusive) {

    int low = 0;
    int high = samples.size();

    // Lower bound: first timestamp >= startInclusive.
    while (low < high) {
        int mid = low + (high - low) / 2;

        if (samples.get(mid).timestampMillis()
                < startInclusive) {
```

Assumptions:

- Samples are sorted by timestamp.
- The input list supports efficient indexed access, as `ArrayList` does.
- The result is materialized in memory, requiring $O(k)$ additional space.

If samples are stored in encoded chunks, the same principle applies at the chunk level: find relevant chunks, then decode the required samples.

2.4 Algorithm: Downsampling and aggregation

Suppose you collect CPU utilization every 5 seconds. A day contains:

$$\frac{24 \times 60 \times 60}{5} = 17{,}280$$

samples per series.

Across 100,000 series, that's 1.728 billion samples per day.

If a dashboard displays one point per minute, it may not need every original sample. A downsampling process can produce aggregates such as:

- Minimum CPU utilization.
- Maximum CPU utilization.
- Average CPU utilization.
- Sample count.

For example, twelve 5-second samples can be aggregated into a 1-minute window.

Averages should be computed using sums and counts:

$$\text{Average} = \frac{\sum_{i=1}^{n} x_i}{n}$$

For aggregation across already-aggregated windows, use the weighted formula:

$$\text{Average} = \frac{\sum_j \text{sum}_j}{\sum_j \text{count}_j}$$

Do not simply average the averages unless all windows contain the same number of observations.

Downsampling is a common technique in long-retention TSDB architectures, but it is not the same as Prometheus's ordinary block compaction. Compaction reorganizes storage; downsampling reduces temporal resolution.

2.5 Algorithm: Compression

Time-series data often exhibits patterns that can be encoded efficiently.

Suppose samples arrive every 10 seconds:

Timestamp	Value
1,000	50.0
1,010	50.1

	1,020	50.1
	1,030	50.2

Instead of storing each timestamp as a full 64-bit integer, an encoding can store a starting timestamp and subsequent deltas:

```
Base timestamp: 1000
Timestamp deltas: 10, 10, 10
Values: 50.0, 50.1, 50.1, 50.2
```

Plain text

Further compression can exploit repeated deltas and similar floating-point bit patterns.

Benefits include:

- Lower disk usage.
- Fewer disk reads.
- Improved memory-cache efficiency.
- Lower bandwidth when transferring data.

The trade-off is CPU consumption during encoding and decoding.

2.6 Complexity summary

Let:

- N = number of series.
- n = samples in one series.
- k = samples matching a range query.
- M = candidate series matching the label filters.

Operation	Illustrative complexity
Exact series lookup	Expected $O(1)$
Find matching series	Depends on index and postings; often related to candidate set sizes
Find range within a sorted series	$O(\log n + k)$
Scan a series range without an index	$O(n)$
Aggregate selected samples	$O(k)$

Append a sample to an in-memory sequence	Amortized $O(1)$, assuming append-friendly storage
Scan all series and samples	Proportional to total data examined

The most important optimization is often not a more sophisticated algorithm. It is **avoiding unnecessary work by filtering series and time ranges early**.

---

3. LLD perspective: Design a mini time-series database

Let's design a simplified TSDB in Java, suitable for an interview or a proof of concept.

3.1 Requirements

Functional requirements:

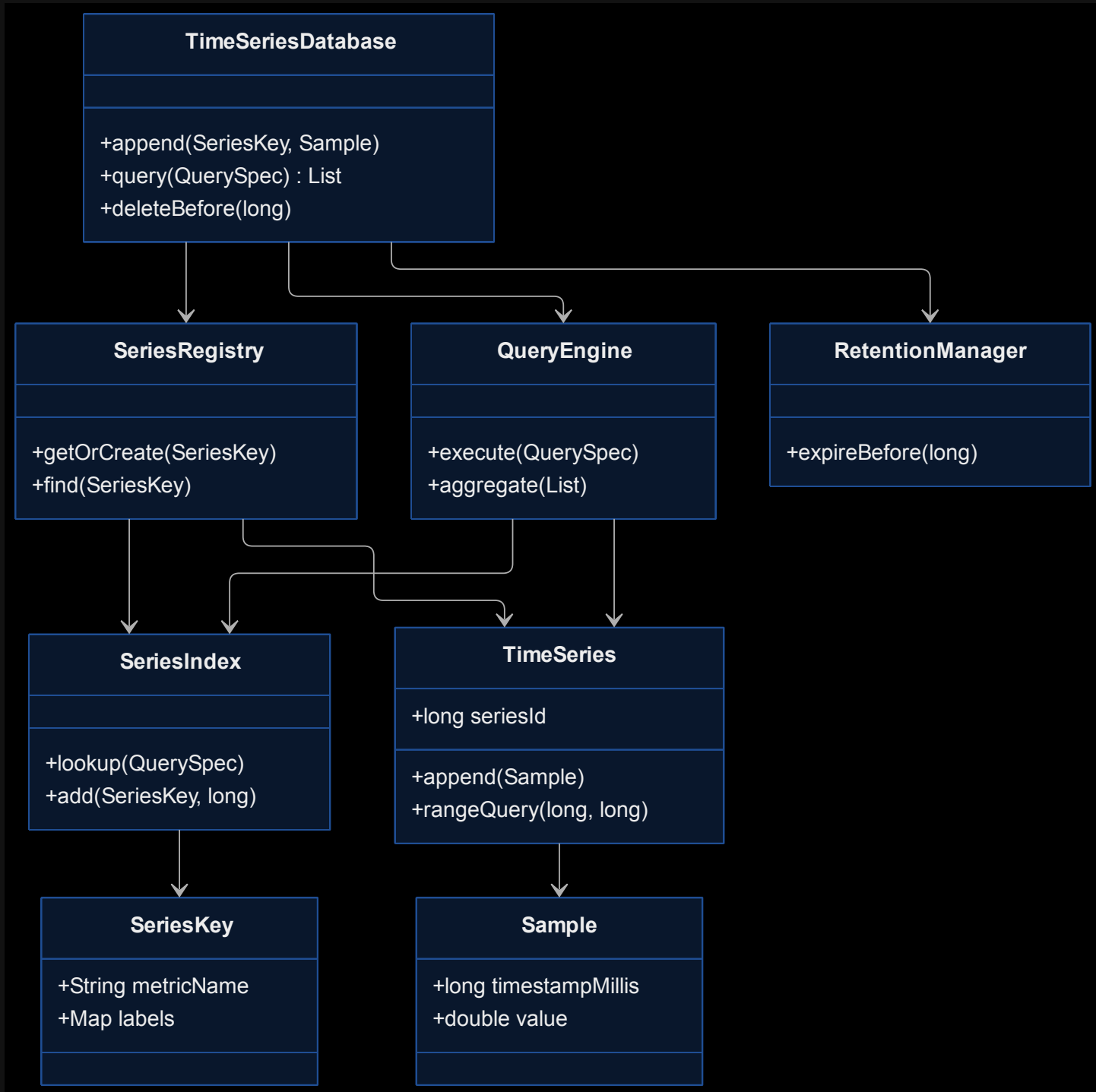
1. Register or identify a series using metric name and labels.
2. Append timestamped samples.
3. Query samples within a time range.
4. Filter by metric name and labels.
5. Calculate aggregations such as average, minimum, and maximum.
6. Enforce a retention policy.

Non-functional requirements:

- High ingestion throughput.
- Low-latency range queries.
- Bounded memory usage.
- Efficient storage.
- Clear concurrency guarantees.
- Extensible storage and indexing strategies.

For this first design, assume a single process, in-memory recent samples, and sorted timestamps per series. Persistence, crash recovery, and distributed storage are later extensions.

3.2 Class diagram



This is a conceptual design. A production implementation would introduce chunk abstractions, persistence, write coordination, and more explicit concurrency control.

3.3 Define the core domain objects

First, define the series identity. The metric name and complete label set must be part of the key.

Java

```
import java.util.Map;
import java.util.TreeMap;
import java.util.Collections;
```

```
record SeriesKey(
    String metricName,
    Map<String, String> labels
) {
    SeriesKey {
        if (metricName == null || metricName.isBlank()) {
            throw new IllegalArgumentException(
                "Metric name is required");
        }

        labels = Collections.unmodifiableMap(
            new TreeMap<>(labels)
        );
    }
}
```

Why sort the labels?

Two keys should compare equal regardless of the insertion order of their label maps.

Plain text

```
{method=GET, status=200}
{status=200, method=GET}
```

These should identify the same series. A canonical representation avoids accidental duplication.

In a production implementation, also validate label names, normalize supported value types, and avoid using a serialized key with ambiguous delimiters.

3.4 Design the time-series abstraction

Java

```
import java.util.List;

interface TimeSeries {
    long seriesId();

    void append(Sample sample);

    List<Sample> rangeQuery(
        long startInclusive,
        long endInclusive
```

```
    );

    void deleteBefore(long timestampMillis);
}
```

This interface hides the storage strategy. We could implement it using:

- An in-memory array or ring buffer.
- Encoded append-only chunks.
- Memory-mapped files.
- A persistent storage engine.

An interface also makes it possible to test query logic independently of storage details.

Important concurrency decision

What happens if one thread appends samples while another queries the same series?

Possible approaches:

- A read-write lock per series.
- Immutable chunks plus a mutable active chunk.
- Copy-on-write snapshots.
- A single-writer model with concurrent readers.

For a simple implementation, a lock is easy to reason about. For a high-throughput TSDB, immutable completed chunks plus a mutable head can reduce contention and simplify concurrent reads.

3.5 Design the series registry and index

The registry resolves a `SeriesKey` to a series object.

Java

```
interface SeriesRegistry {
    TimeSeries getOrCreate(SeriesKey key);

    TimeSeries find(SeriesKey key);
}
```

However, an exact-key map alone cannot efficiently answer:

Plain text

```
Find all series where service="payment".
```

Therefore, add an inverted index:

Java

```java
interface SeriesIndex {
    void add(SeriesKey key, long seriesId);

    void remove(SeriesKey key, long seriesId);

    // Return candidate IDs matching metric and labels.
    java.util.Set<Long> lookup(
        QuerySpec query
    );
}
```

Internally, one possible structure is:

Plain text

```
label name → label value → set of series IDs
```

When a query has multiple filters, retrieve the candidate sets and intersect the smallest sets first.

For example:

Plain text

```
service="payment" → 100,000 series
status="500"      → 2,000 series
region="ap-south" → 300 series
```

Intersecting the smaller candidate sets first can reduce intermediate allocations and processing.

3.6 Design the query model

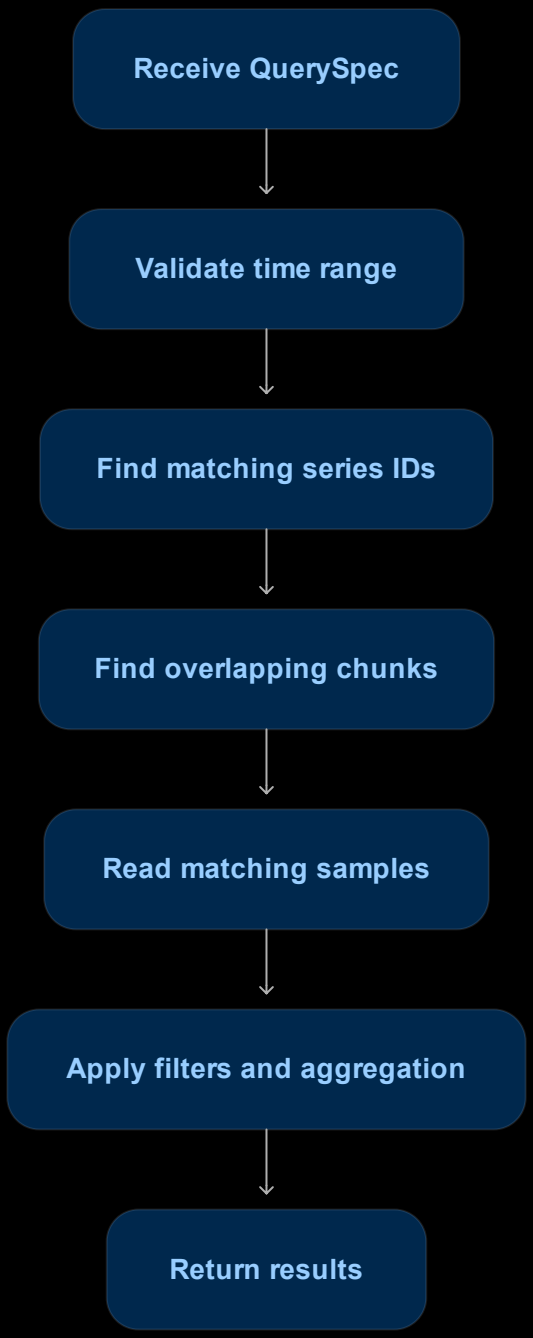
Java

```
import java.util.Map;

record QuerySpec(
    String metricName,
    Map<String, String> exactLabels,
    long startInclusive,
    long endInclusive
) {
    QuerySpec {
        exactLabels = Map.copyOf(exactLabels);

        if (startInclusive > endInclusive) {
            throw new IllegalArgumentException(
                "Start must not exceed end");
        }
    }
}
```

A query engine can then follow this flow:



Notice the separation of concerns:

- The index identifies relevant series.
- The storage layer retrieves samples.
- The query engine applies time-series operations.
- The API layer handles HTTP requests and serialization.

3.7 API design

A simplified ingestion endpoint could look like this:

```http
POST /api/v1/samples
```

```
Content-Type: application/json
```

JSON

```
{
  "metric": "cpu_usage_percent",
  "labels": {
    "service": "payment",
    "instance": "payment-01"
  },
  "timestampMillis": 1792852800000,
  "value": 67.4
}
```

A corresponding query endpoint:

http

```
GET /api/v1/query_range
    ?metric=cpu_usage_percent
    &start=1792852500000
    &end=1792852800000
```

This is an illustrative API, not Prometheus's exact API contract. A production API should define timestamp units, limits, validation, authentication, error responses, and pagination or streaming for large results.

For bulk ingestion, prefer batching samples rather than issuing one network request per sample.

3.8 Testing strategy

At the LLD level, test both functional correctness and operational behavior.

Test case	Expected behavior
Append first sample	Creates or resolves the series
Append another sample	Adds a new point
Query empty range	Returns no samples
Query partial range	Returns only matching timestamps

Reordered label insertion	Resolves to the same series key
Query multiple label filters	Returns only matching series
Expire old samples	Removes samples older than the retention boundary
Concurrent append/query	Does not expose corrupted or inconsistent data
Duplicate timestamp	Follows the explicitly defined policy
Out-of-order sample	Accepted, rejected, or reconciled according to policy

Design decision: Never leave duplicate timestamps, out-of-order writes, or concurrent visibility as accidental behavior. Define their semantics explicitly before implementing the storage engine.

---

4. HLD perspective: Design a scalable monitoring platform

Now let's move from one database instance to a production monitoring platform similar to Prometheus.

4.1 Requirements and assumptions

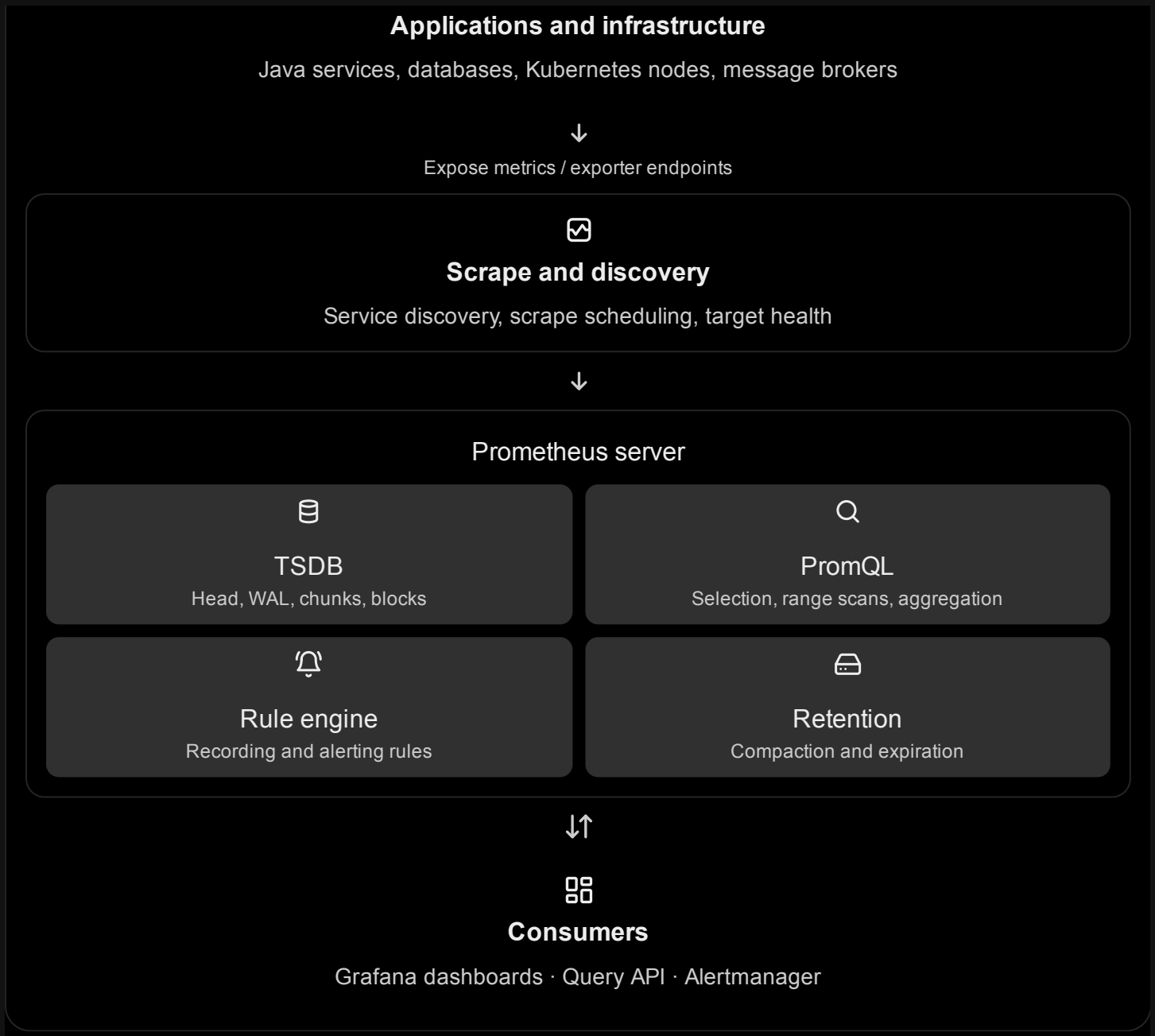
Assume we are designing monitoring for a distributed system with 10,000 application instances.

Requirement	Design objective
Ingestion	Collect metrics continuously
Querying	Support instant queries and historical range queries
Alerting	Detect high error rates and resource exhaustion
Retention	Retain recent raw samples and configurable historical data
Availability	Continue monitoring through selected component failures
Scalability	Support increasing targets, samples, and query traffic
Efficiency	Reduce storage cost through compression and retention
Isolation	Prevent expensive queries from overwhelming ingestion

A key workload characteristic is that metric ingestion is generally append-heavy, while reads frequently scan time windows across multiple series.

4.2 High-level architecture





This is a logical architecture; it does not imply every component is a separate process. Prometheus includes several of these functions within a single server. Its local TSDB is not, by itself, a clustered, replicated database.

Prometheus +2

4.3 Write path: From application to disk

Let's trace a single sample through the system.

Suppose a Java application exports:

Plain text

```
http_requests_total{
    service="payment",
```

```
    instance="payment-01",
    status="500"
} 27
```

1. **Expose metrics.** The application or exporter makes the current metric values available through an HTTP endpoint.
2. **Discover and scrape.** Prometheus discovers the target and periodically fetches its metrics. This is a pull-based ingestion model.
3. **Identify the series.** The metric name and label set determine the series identity.
4. **Append the sample.** The timestamp and value are added to the current in-memory head.
5. **Protect recent data.** The write-ahead log (WAL) allows recovery of recent ingested data after a restart, subject to the actual durability and filesystem guarantees.
6. **Persist and compact.** The head is persisted into time-based blocks; background compaction reorganizes those blocks for longer-term storage.

Prometheus's documented local storage layout uses two-hour blocks, an index, chunks, metadata, and a WAL. Recent data is held in the head before becoming part of persisted blocks.

Prometheus +1

Why use both memory and disk?

Writing every sample as an independent database transaction with multiple random disk updates would introduce unnecessary overhead.

Instead, the storage design combines:

- Fast ingestion into an active head.
- Sequential WAL writes for recovery.
- Compact, encoded sample chunks.
- Immutable historical blocks.
- Background compaction.

This design reduces the cost of repeated random updates and makes historical data easier to scan.

4.4 Read path: Execute a PromQL query

Suppose we want the HTTP request rate over five minutes, grouped by service.

```promql
```

```
sum by (service) (
  rate(http_requests_total[5m])
)
```

This expression is illustrative of a standard PromQL query.

Conceptually, the engine:

1. Parses and validates the expression.
2. Finds series matching the metric selector.
3. Selects the required historical samples for each series.
4. Computes counter rates over the requested window.
5. Aggregates the resulting rates by `service`.
6. Returns the resulting time series.

The `rate()` function calculates the per-second increase of a counter, accounting for counter resets. It should generally be used with counter metrics rather than gauges.

Prometheus +1

An instant query evaluates an expression at one point in time. A range query evaluates it at successive timestamps over a time interval. Consequently, a range query can be much more expensive than its final number of output series suggests.

Prometheus

Query optimization

For large datasets:

- Filter by metric and selective labels early.
- Restrict the time range.
- Avoid unnecessary high-cardinality dimensions.
- Limit the number of returned series.
- Use recording rules for frequently repeated expensive expressions.
- Apply query concurrency and timeout limits.

A query that returns ten aggregated series may still scan millions of underlying samples. Output size is not a reliable measure of query cost.

4.5 Storage engine: Head, WAL, chunks, and blocks

Head block

Contains recent active series and samples, with in-memory structures for ingestion and querying.

Write-ahead log

Records recent changes for crash recovery. It is not a substitute for a complete backup strategy.

Time-based blocks

Persisted data organized into chunks, indexes, and metadata. Historical queries read relevant blocks.

Compaction and retention

Compaction creates larger blocks; retention removes expired data according to configured limits.

These are the actual major concepts in Prometheus's local storage architecture, although implementation details vary by version.

Prometheus +1

A subtle but important distinction:

- **WAL:** supports recovery of recent writes.
- **Compaction:** reorganizes stored data into larger blocks.
- **Retention:** removes old data.
- **Replication or remote storage:** addresses requirements beyond a single local storage instance.

These solve different problems.

4.6 Capacity planning: How much storage do we need?

The primary sizing inputs are:

- T: number of active time series.
- I: scrape interval in seconds.
- D: retention duration in seconds.
- B: average bytes per sample, including a suitable allowance for storage overhead.

The approximate ingestion rate is:

$$R = \frac{T}{I}$$

The raw sample count retained over the period is approximately:

$$N = \frac{T}{I} \times D$$

And the estimated sample storage is:

$$S = \frac{T}{I} \times D \times B$$

Prometheus documents a rough average of 1–2 bytes per sample for capacity estimation, but real requirements depend on the workload, series churn, indexes, WAL, head memory, and operational overhead.

Prometheus

Consider a system with 100,000 active series, a 15-second scrape interval, and 15 days of retention.

Active series	Samples/second
100,000	**6,667**
Samples/day	Samples/15 days
576 million	**8.64 billion**

At 1–2 bytes per sample, the rough sample-storage estimate is 8.64–17.28 GB in decimal units. This is not a complete disk-capacity estimate.

Production capacity must include index and metadata overhead, WAL and head requirements, compaction's temporary space requirements, query working memory, and a safety margin.

For a single Prometheus server, retention size should leave room for compaction. The official documentation recommends keeping the configured retention size at no more than approximately 80–85% of allocated disk capacity.

Prometheus

4.7 The biggest scaling challenge: Cardinality

In Prometheus, **cardinality is the number of distinct time series**, not the number of metric names.

Consider this metric:

Plain text

```
http_requests_total{
  service="payment",
  status="500",
  instance="payment-01"
}
```

Suppose you have:

- 20 services.
- 5 HTTP status categories.
- 10 instances per service.
- 4 HTTP methods.

The approximate number of series for this metric is:

$$20 \times 5 \times 10 \times 4 = 4{,}000$$

Now imagine adding a `user_id` label with one million distinct users. In the worst case, this can multiply the number of series dramatically.

Other dangerous labels include:

- Request IDs.
- Session IDs.
- Raw URLs containing unique parameters.
- Unbounded customer identifiers.

Why high cardinality is expensive

Each distinct series needs identity and indexing information, as well as storage for its samples. Creating and removing many short-lived series also adds churn.

This affects:

- Memory consumption.
- Index size.
- Ingestion performance.
- Query planning and execution.
- CPU usage and garbage collection.

The correct approach is usually to keep metric labels bounded and move high-cardinality diagnostic information to a suitable logs or traces backend.

For example:

```
Plain text

Good:
http_requests_total{route="/orders/{id}", status="500"}

Risky:
http_requests_total{url="/orders/ORD-9837291", status="500"}
```

The first groups requests by a bounded route template; the second can create a separate series for every unique order ID.

This is a crucial monitoring-system design decision. Increasing server capacity does not eliminate an uncontrolled cardinality problem.

4.8 Scaling ingestion and queries

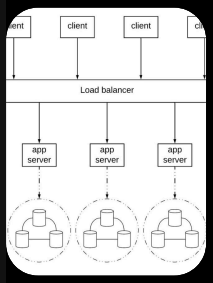
There are several ways to scale a TSDB, depending on the workload.



Vertical scaling

Give the server more CPU, RAM, and fast local SSD storage.

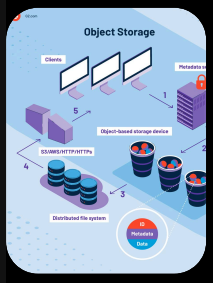
Best for: simpler deployments where a single server still has sufficient capacity.



Horizontal partitioning

Distribute series across multiple ingestion and storage nodes. Queries may need to fan out to several partitions and merge the results.

Best for: workloads that exceed the capacity of a single node.



Remote or long-term storage

Keep recent samples locally and send data to a remote backend for longer retention or centralized querying.

Best for: longer history, cross-cluster queries, and larger monitoring environments.



Read-path optimization

Cache dashboard queries, use recording rules, isolate expensive analytical workloads, and limit query concurrency.

Best for: reducing contention between ingestion and dashboard traffic.

These are general TSDB scaling strategies. Prometheus's local storage is not a distributed cluster by itself; horizontal scaling requires an appropriate deployment architecture or an external system.

Prometheus +2

A note on high availability

Running two Prometheus servers scraping the same targets can provide independent monitoring paths. However, duplicate collection is not the same as a replicated database with automatic failover and a single strongly consistent view.

A production design must explicitly consider:

- Independent failure domains.
- Duplicate samples and query deduplication.
- Whether alerting rules run on both servers.
- Remote storage availability.
- Data gaps during outages.
- Recovery and retention guarantees.

If you need durable, globally queryable, long-term metrics, evaluate a remote-storage or distributed metrics architecture against your consistency, availability, and cost requirements.

---

5. LLD meets HLD: Important design decisions

These are the trade-offs I would expect a senior or staff-level engineer to explain during a system design interview.

5.1 Pull versus push ingestion

Dimension	Pull model	Push model
Initiator	Collector	Application or agent
Target discovery	Central collector discovers targets	Sender needs a destination
Health monitoring	Collector can detect scrape failures	Requires explicit sender/agent health mechanisms

Short-lived jobs	Can be difficult to scrape before termination	Often easier to push completed-job metrics
Network topology	Collector must reach targets	Sender can initiate outbound connections
Backpressure	Controlled partly by scrape scheduling	Depends on queueing and sender behavior

Prometheus primarily uses a pull model, while supporting push-based workflows for certain use cases, such as short-lived batch jobs through Pushgateway.

The choice is not simply about performance. It affects discovery, failure detection, networking, security, and delivery semantics.

5.2 Counter, gauge, histogram, and summary

The metric type determines which algorithms and queries are appropriate.

Type	Example	Typical query
Counter	Total requests	`rate(http_requests_total[5m])`
Gauge	Current queue depth	`avg_over_time(queue_depth[5m])`
Histogram	Request duration distribution	`histogram_quantile(...)` for classic histogram buckets
Summary	Client-calculated quantiles and observation statistics	Query exported quantiles and count/sum metrics

A counter typically increases and may reset. A gauge can increase or decrease. Histograms and summaries are intended to describe distributions of observations, such as request latency.

Prometheus +1

For example, if the application reports:

Plain text

```
http_requests_total = 1,000
```

at one scrape and `1,300` at the next scrape, the counter increased by 300. But if the application restarted and the counter reset, interpreting the difference as a negative rate would be incorrect. Prometheus's `rate()` function accounts for counter resets.

For latency, averaging alone is insufficient. A mean latency of 100 ms could hide a small percentage of requests taking several seconds. Histograms allow percentile-oriented analysis when appropriately configured.

5.3 Retention and consistency semantics

A monitoring TSDB must explicitly handle several cases:

- Samples arriving out of order.
- Repeated timestamps.
- Scrape failures and missing data.
- Counter resets.
- Series becoming stale.
- Queries overlapping active ingestion.
- Deletion and retention cleanup.

Do not assume that every TSDB accepts arbitrary out-of-order writes or handles duplicates identically. Those semantics depend on the engine and its configuration.

Similarly, a monitoring database is not necessarily a transactional database. You typically optimize for efficient ingestion and analytical reads rather than general-purpose multi-row transactions.

---

6. Putting it together: End-to-end example

Imagine a Spring Boot payment service running on Kubernetes.

Step 1: Instrument the service
Expose request counts, latency, error counts, JVM memory, and thread metrics.

Step 2: Collect samples
Prometheus discovers the pod and scrapes its metrics endpoint every 15 seconds.

Step 3: Store and index
The TSDB resolves series identities, appends samples, and persists data through its storage lifecycle.

Step 4: Query and aggregate
PromQL calculates request rates, error ratios, latency percentiles, and resource trends.

Step 5: Alert

An alerting rule detects elevated error rates and sends notifications through Alertmanager.

For example, an error ratio query could be:

```promql
sum by (service) (
  rate(http_requests_total{status=~"5.."}[5m])
)
/
sum by (service) (
  rate(http_requests_total[5m])
)
```

This computes the proportion of requests returning 5xx responses over the five-minute rate window, grouped by service.

An alert could fire when the ratio exceeds a threshold for a sustained duration. In practice, also guard against division by zero, insufficient traffic, missing metrics, and incorrect label matching.

7. Interview preparation: What to be able to explain

Use this checklist to assess your understanding.

Readiness checklist 0 / 15 completed

DSA

- [ ] Explain series identity and label indexing.
- [ ] Implement a binary-search-based range query.
- [ ] Explain delta encoding and chunk compression.
- [ ] Analyze query complexity and aggregation cost.
- [ ] Explain why cardinality affects memory and query performance.

LLD

- [ ] Design SeriesKey, Sample, TimeSeries, and SeriesIndex.
- [ ] Define append, range-query, and retention interfaces.
- [ ] Specify concurrent read/write semantics.
- [ ] Handle duplicate and out-of-order samples.

- [ ] Write unit tests for range queries and retention.

HLD

- [ ] Draw the scrape-to-storage-to-query flow.
- [ ] Explain the head, WAL, chunks, blocks, and compaction.
- [ ] Estimate storage from series count, scrape interval, and retention.
- [ ] Design for ingestion growth and expensive queries.
- [ ] Discuss HA, remote storage, and failure recovery.

Copy checklist

8. Further reading

The official documentation is the best starting point for understanding implementation details rather than relying on generic TSDB descriptions.

- Prometheus storage documentation — Storage architecture, WAL, blocks, and retention ↗.
- Prometheus data model — Metric names, labels, series, and samples ↗.
- PromQL basics — Instant queries, range queries, and selectors ↗.
- Prometheus histograms and summaries — Latency distributions and histogram queries ↗.

Final takeaway

Think of a TSDB as three cooperating layers:

- **DSA:** Efficient series indexing, timestamp-ordered sample access, compression, and aggregation.
- **LLD:** Explicit series identity, storage abstractions, query execution, concurrency, and retention semantics.
- **HLD:** Scraping, durable ingestion, storage lifecycle, distributed query execution, high availability, and capacity management.

For a staff-level system design interview, the strongest answers go beyond describing a database's components. They explain **why the data structures fit the workload, how the write and read paths interact, where cardinality becomes a bottleneck, and what trade-offs are necessary to meet throughput, latency, durability, and cost requirements**.